

【Step 3 – Branch graph】中将骨架体素进行压缩，不会丢失一些拓扑几何信息，从而影响后续神经网络的学习吗？

 ChatGPT gpt-5-6-thinking

会。如果“压缩”理解成把一串骨架体素直接删掉，只留下两个端点并用直线连接，那么一定会丢失重要几何信息。但这不是我建议的 Branch Graph。

更准确的做法是：

压缩的是“图中的节点数量”，不是删除原始几何。

也就是说，Branch Graph 应该是一种带几何载荷的图（**geometry-aware branch graph**）：拓扑层面把大量没有分叉意义的中间体素合并成一条 branch；但这条 branch 内部仍然保存原来的完整中心线路径、沿程管径等信息。

这点对我们现在的课题非常重要。

1. 先看“错误的压缩”是什么

假设骨架化以后，一根弯曲血管有很多点：

text



如果简单压缩成：

text

A ----- B

只保留 A、B 两点。

那确实会丢掉：

- 血管实际长度；

- 曲折程度；
- 曲率；
- 三维方向变化；
- 沿程半径变化；
- 局部扩张/收缩。

这种表示不能用于我们的 Geometry Generator，也不适合用来研究真实血管形态规律。

2. 我建议的 Branch Graph 不是这样

真正的做法是：

```
text
```

图结构层面：

```
A ===== B
      Branch 17
```

看起来只有：

```
A — Branch 17 — B
```

但是 Branch 17 内部保存：

```
text
```

```
centerline:
```

```
P1 → P2 → P3 → ... → P47
```

```
radius:
```

```
r1 → r2 → r3 → ... → r47
```

甚至可以保存：

```
text
```

```
curvature(s)
```

```
cross-section(s)
```

```
local direction(s)
```

所以：

47 个骨架节点从“图节点”变成了“Branch 17 的内部几何数据”。

信息没有必要消失。

3. 一个很好的类比：高速公路地图

假设一条高速公路沿途有 1000 个 GPS 点。

原始数据：

text

● ● ● ● ● ● ● ● ● ● ● ● ● ● ● ● ● ● . . .

但做交通网络分析时，没必要把每一个 GPS 点都当作“交通节点”。

真正有意义的是：

text

城市 A ===== 高速公路 ===== 城市 B

图里只有两个城市。

但是高速公路这个 edge 内部仍保存完整 GPS 轨迹：

text

```
lon1, lat1
lon2, lat2
...
lon1000, lat1000
```

所以：

网络结构被简化了，公路形状没有被扔掉。

Branch Graph 对血管做的就是同样的事情。

4. 从数学上说，哪些骨架点可以放心“收起来”？

对于普通树状区域:

text

中间节点如果都是：

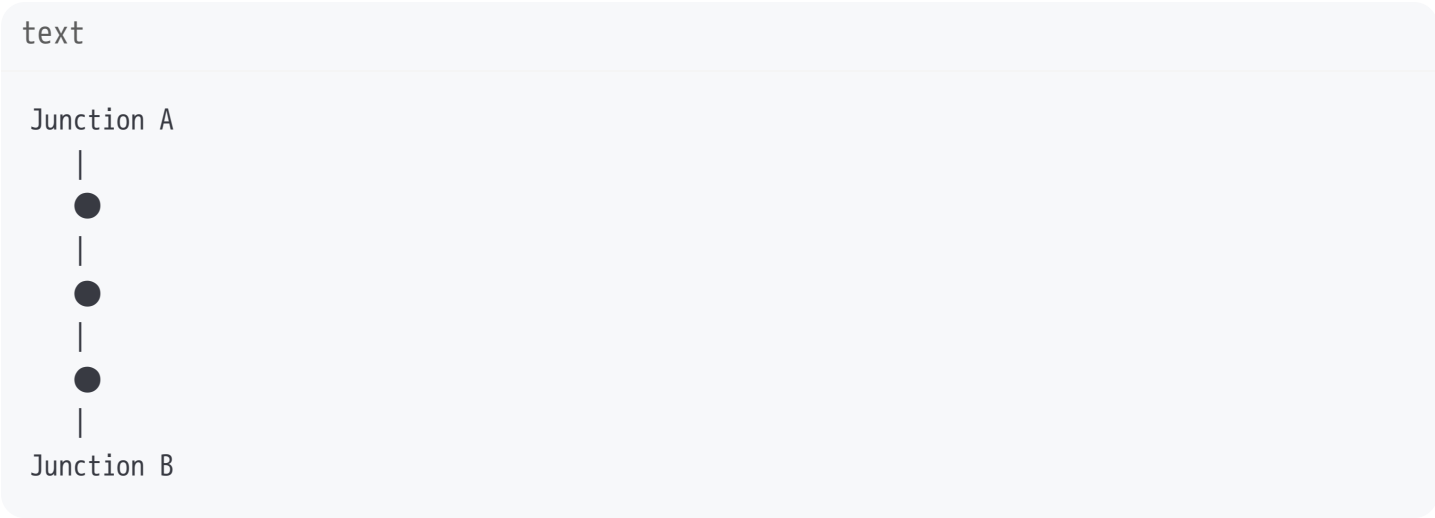
一个上游邻居 + 一个下游邻居

也就是没有发生：

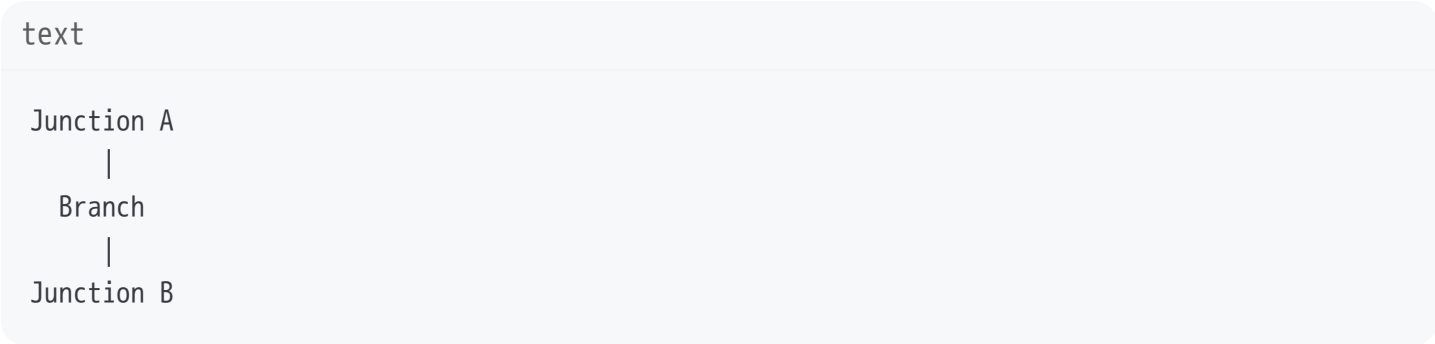
- 分叉；
- 汇合；
- 终止；

那么这些点对于“**谁和谁连接**”这个问题其实没有新的信息。

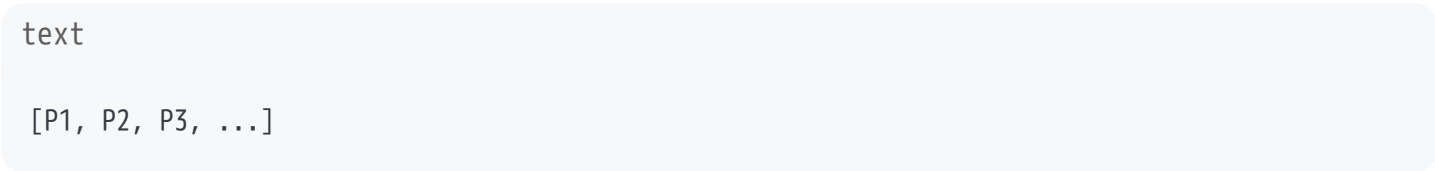
所以可以把：



变成：



只要 Branch 中仍保存：

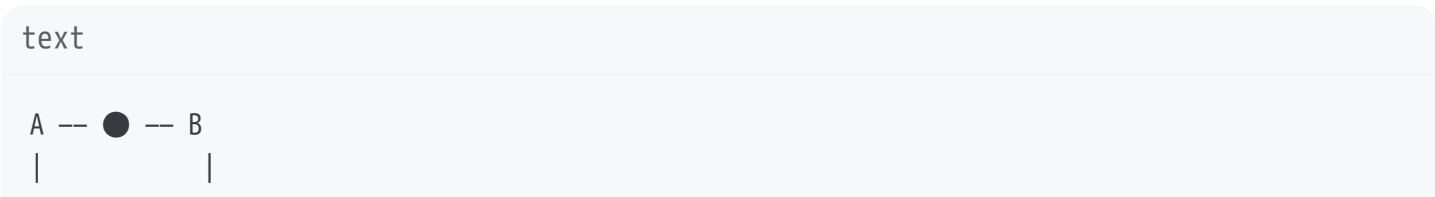


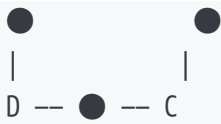
这种压缩在拓扑意义上基本不会改变 connectivity。

5. 但有一个例外：环结构必须特殊处理

这一点对脑血管尤其重要。

例如：





如果所有环上的中间节点度都是 2，而算法简单粗暴地说：

“所有 degree=2 的节点全部删除。”

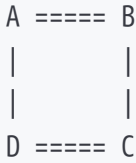
可能会把整个 cycle 处理坏。

所以 Branch Graph 构建必须明确：

树状 chain 可以收缩，但 cycle 必须保留 cycle identity。

例如表示成：

text



或者至少保留一个 cycle anchor。

因此以后如果处理：

- Circle of Willis;
- 毛细血管网;
- 吻合支;

要专门做 cycle-preserving graph contraction。

6. 更重要的是：真正最大的误差来源其实不是“Branch Graph 压缩”

而是前一步：

Skeletonization

例如原 STL 是：

text



体素化以后：

text



再 skeletonize:

text



如果体素分辨率过低，就可能发生：

小血管消失

text

真实：



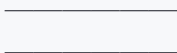
骨架：



两根靠得很近的血管错误粘在一起

text

真实：



骨架：



一个真实分叉被识别成多个假分叉

或者：

一个真实环被切断

所以我们真正需要控制的是：

STL → voxel → skeleton 这一步的拓扑误差。

Branch contraction 本身反而容易做到可逆或近似无损。

7. 这也是为什么我之前建议“粗图只负责导航”

对于你那种一大坨完整血管，我现在会把数据分成两个层级。

Level A: Whole-network coarse graph

从完整 STL 做：

```
text

STL
↓
voxel
↓
skeleton
↓
coarse branch graph
```

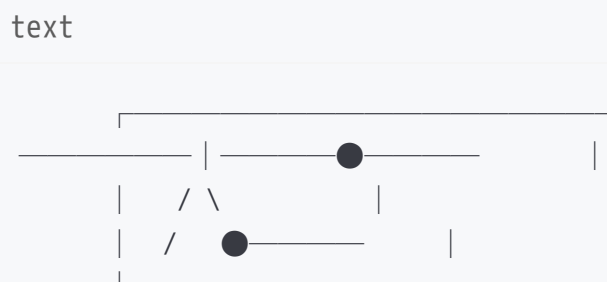
它的目的只是：

知道整张网络大概怎么连，并找到值得裁剪的局部区域。

它不是以后 Geometry Generator 的最终训练真值。

Level B: Local high-fidelity graph

从 coarse graph 找到一个局部 patch：



然后：

重新回到原始 STL。

把这一片真实表面裁出来。

再对小模型进行高质量：

```
text

local STL
↓
精确 centerline
```

↓
精确 radius profile
↓
Local Vascular Centerline Graph

这份数据才真正送给：

- 可解释 GNN；
- Geometry Generator；
- CFD。

这样，全局粗 **skeleton** 的几何误差就不会直接污染最终训练数据。

8. 所以现在建议采用“双表示”

这一点我认为可以直接作为你的数据架构。

表示 1：Branch-level Graph

给：

Topology / Interpretable GNN

看。

例如：

```
text

Node A
  |
Edge 17
  |
Node B
  /   \
Edge18 Edge19
```

每条 edge 附带：

```
text

length
mean caliber
tapering
tortuosity
branch order
subtree size
...
```


它解决的是：

“这条血管在网络中是什么角色？”

表示 2: Branch Geometry Sequence

给：

Geometry Generator

看。

对于 Edge 17：

```
text

P1: x,y,z,r
P2: x,y,z,r
P3: x,y,z,r
...
Pk: x,y,z,r
```

它解决：

“这根 branch 具体怎么弯、怎么变粗变细？”

因此：

```
text

Branch Graph
≠
把 geometry 扔掉
```

而是：

```
text

Branch Graph
+
Branch Geometry Payload
```

两者一起才是完整的：

Vascular Centerline Graph

9. 这恰好和我们现在的两个神经网络任务对应

可解释 GNN

不一定需要看到每条血管的 100 个点。

它研究的是：

parent-daughter 规律。

所以可以给它：

```
text

Branch 17:

代表尺度
长度
tortuosity
taper
branch order
subtree size
...
```

这反而比塞给它 100 个 XYZ 更容易解释。

否则 symbolic regression 最后可能面对几百维坐标，很难发现有意义规律。

10. Geometry Generator 则不能只看压缩统计量

Geometry Generator的任务是生成：

连续三维中心线 + 连续 local caliber profile。

所以训练时必须看到完整 branch geometry。

例如：

```
text

Graph context
  ↓

Branch 17 应该总体：
length ≈ ...
scale ≈ ...
direction ≈ ...

  ↓

Geometry Generator
```



P1,r1
P2,r2
...
P32,r32

最后 spline:

text

离散点



连续 centerline + r(s)

因此:

GNN规律发现使用“压缩后的 branch-level 特征”； Geometry Generator 使用“branch 内部完整几何序列”。

这两个数据层次必须同时保存。

11. 甚至最好不要只保存“平均半径”

例如真实一根血管:

text

distance →
radius:

1.80
1.77
1.69
1.72
1.66
1.59
1.55

如果 Branch Graph 只写:

text

mean radius = 1.68

那么确实丢失了:

沿程变化。

所以建议 edge 至少同时保存：

给可解释 GNN 的 **summary**

text

mean / median caliber
proximal caliber
distal caliber
taper rate
radius CV

以及：

给 **Geometry Generator** 的原始/平滑 **profile**

text

[r1, r2, ..., rk]

这叫：

summary + sequence

而不是二选一。

12. 中心线也一样

可解释 GNN 不一定需要：

text

32 × XYZ

它可以看：

text

length
straight-line distance
tortuosity
mean curvature
max curvature
initial direction
end direction

但是 Geometry Generator 仍然保存：

text

P1
P2
...
P32

所以数据结构可以是：

text

```
Branch 17
|
|—— Topology
|   parent
|   children
|
|—— Summary features
|   length
|   caliber
|   tortuosity
|   taper
|   curvature...
|
|—— Dense geometry
    P1,r1
    P2,r2
    ...
    Pk,rk
```

这个结构非常适合你的研究。

13. 为什么这种设计反而对“可解释 GNN”更好？

假如完全不压缩骨架：

text

一个 20 mm 的 branch
↓
可能有 200 个 voxel nodes

GNN会看到：

text

node1
node2

```
node3
...
node200
```

而其中：

```
text
```

```
node 37
node 38
node 39
```

可能根本没有生物学上的“事件”。

这会造成：

- 图非常大；
- message passing 非常长；
- 分叉关系反而被大量中间点淹没；
- symbolically interpreting messages 变困难。

而压成：

```
text
```

```
Parent branch
  ↓
Bifurcation
  /   \
Child A Child B
```

后：

模型的每一次 message passing 都对应更加明确的血管学关系。

这对于我们要参考 Cranmer 的：

GNN message → Symbolic Regression

尤其重要。

因为我们希望 message 有机会被解释成：

“父支向子支传递了什么形态信息”。

而不是：

“第 138 个 voxel 向第 139 个 voxel 传了什么”。

14. 我会进一步调整“可解释 GNN”的图定义

之前可以想成：

junction = node, branch = edge。

但为了规律解释，我认为还可以考虑：

Branch-as-node graph

也就是：

text

原血管：

Branch A

↓

bifurcation

/

\

Branch B

Branch C

转换为：

text

[Branch B]

/

[Branch A]

\

[Branch C]

即：

一根完整 branch 就是一个 GNN node。

每个 node 特征：

text

length

caliber

taper

tortuosity

curvature

orientation

...

连接：

text

$A \rightarrow B$

$A \rightarrow C$

表示：

parent–daughter relationship。

这个表示可能特别适合你的“规律发现”。

15. 为什么 Branch-as-node 可能比 centerline-voxel graph 更适合 Cranmer 范式？

因为你的科学问题本来就是：

一根 parent branch 与 daughter branch 之间是什么规律？

所以：

text

GNN message:

Branch A $\rightarrow$ Branch B

就可以直接解释成：

父血管对这个子血管的结构关系。

然后你可以对：

text

message($A \rightarrow B$)

做 symbolic regression。

这比：

text

voxel 817 $\rightarrow$ voxel 818

有意义得多。

所以压缩不仅不一定伤害科学问题，

反而可能帮助可解释性。

16. 但要防止“过度压缩”

有三种信息我明确不建议扔掉。

① Branch geometry sequence

必须保留：

```
text
```

```
P(s), r(s)
```

供 Geometry Generator 使用。

② Junction-local geometry

分叉附近最好额外保存：

```
text
```

```
parent incoming direction  
daughter outgoing directions  
bifurcation angles  
local caliber
```

因为这是很多规律可能发生的地方。

③ Global / downstream metadata

即使裁成 local patch，也保留：

```
text
```

```
original subtree size  
depth  
terminal count  
territory
```

否则可解释 GNN 只能学纯局部关系，无法判断拓扑上下文到底有没有作用。

17. 所以我会把整个数据结构定义成三个尺度

这是我最推荐你记住的版本。

Scale 1 — Topology

回答：

谁和谁连接？

数据：

text

parent
children
cycle
branch order
subtree size

Scale 2 – Branch morphology

回答：

整根 branch 总体长什么样？

数据：

text

length
representative caliber
taper
tortuosity
orientation
curvature statistics

Scale 3 – Dense local geometry

回答：

沿这根血管每一个位置具体长什么样？

数据：

text

$P(s)$
 $r(s)$
optional cross-section(s)

然后：

可解释 GNN主要工作在 Scale 1+2。

这就避免“为了压缩图而丢掉几何”。

18. 所以 Step 3 的名字甚至应该改

之前写：

Step 3 – Branch Graph：把骨架压缩

容易让人误以为：

把数据扔掉。

我建议以后改成：

Step 3 – Hierarchical Vascular Representation

中文：

建立分层血管表示。

过程是：

```
text

Voxel skeleton
  ↓
识别 junction / terminal / cycle
  ↓
聚合 degree-2 chains
  ↓
Branch-level topology
  +
保留每个 branch 的完整 centerline geometry
```

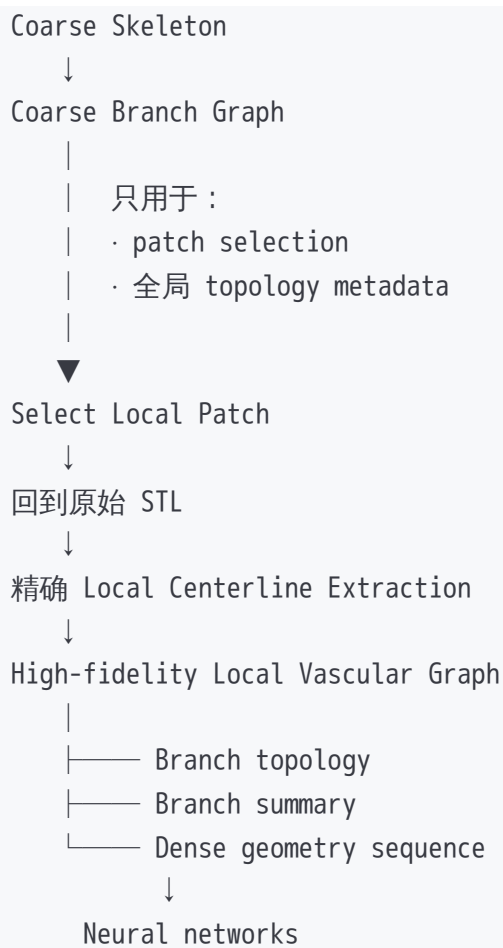
“聚合”比“压缩”更准确。

19. 而且你的全局粗图和局部精细图要明确区分

现在完整数据流程最好写成：

```
text

Whole STL
  ↓
Voxelization
  ↓
```



所以：

粗 skeleton 的任何小几何损失，不直接进入最终 Geometry Generator 的 ground truth。

这是避免你担心问题的最关键措施。

20. 真正需要做的一个验证实验

你的担忧完全应该被实验验证，而不是凭感觉说“不丢”。

以后 preprocessing 应该做一个：

Representation Fidelity Test

随机选比如几十个 local patches。

比较：

text

Original STL

vs

Branch Graph reconstructed geometry

至少检查：

- connectivity 是否一致；
- bifurcation count 是否一致；
- cycle count 是否一致；
- centerline length error；
- tortuosity error；
- local radius profile error；
- branch angle error。

如果：

topology 改变，

那说明 skeleton/graph extraction 不可靠。

如果：

topology 一致，但 geometry summary 偏差大，

说明采样/平滑参数需要调整。

21. 还可以做“分辨率敏感性”

例如把同一 STL：

```
text
```

```
voxel size = h1  
voxel size = h2  
voxel size = h3
```

分别 skeletonize。

然后看：

```
text
```

```
branch number  
junction number  
cycle number  
length  
tortuosity
```

是否稳定。

如果某些小血管随着 voxel size 稍变就：

那它们本身就处在：

当前成像/网格无法稳定解析的尺度。

这时与其让 GNN 学这些不稳定结构，

反而应该：

设定最小可靠血管尺度阈值。

这会让你的数据更科学。

22. 最终直接回答你的问题

“将骨架体素压缩成 Branch Graph，会不会丢失拓扑几何信息、影响 GNN？”

答案要分两部分。

对拓扑：

如果只收缩非分叉的 degree-2 chain，并对 cycle 特殊处理，理论上可以保留主要 connectivity；真正更危险的是前面的 skeletonization 产生假断裂、假连接或漏掉细血管。

对几何：

如果只保留 branch 两个端点和几个平均统计量，会严重丢失信息；如果把中心线点序列、沿程管径 profile 等作为 branch 的内部数据保存，则完全没有必要把这些信息扔掉。

而且对我们的研究：

Branch-level 聚合反而有利于可解释 GNN，因为它让 GNN message 对应“父支 → 子支”这种真正有生物学意义的关系。

因此我建议最终采用：

Hierarchical Vascular Graph = branch-level topology + branch-level interpretable features + dense centerline/radius geometry。

这样既避免超大 voxel graph 带来的计算负担，又不会牺牲 Geometry Generator 所需的几何信息，同时还能让后续 Interpretable GNN → Symbolic Regression 的解释对象变得更加明确。